

WIFI Network Basics

WIFI Network Basics

1. Electromagnetic Waves and Wireless Communication
 - 1.1 What are Electromagnetic Waves
 - 1.2 Why WiFi Uses 2.4GHz
2. How WiFi Works
 - 2.1 Channels
 - 2.2 RSSI (Signal Strength)
 - 2.3 802.11 Protocol Evolution
 - 2.4 WiFi Security Authentication
3. WiFi Operating Modes
 - 3.1 STA Mode (Station)
 - 3.2 AP Mode (Access Point)
 - 3.3 STA + AP Hybrid Mode
 - 3.4 STA vs AP Comparison
4. ESP32-S3 WiFi Hardware Specifications
5. ESP32-S3 WiFi Development Key Points
 - 5.1 WiFi and ADC2 Resource Conflict
 - 5.2 NVS - WiFi Storage Dependency
 - 5.3 WiFi Initialization Fixed Sequence
 - 5.4 Antenna
 - 5.5 Strapping Pins
 - 5.6 WiFi Event-Driven Model
 - 5.7 EventGroup Synchronization Mechanism
6. WiFi OTA Firmware Upgrade
 - 6.1 What is OTA
 - 6.2 Flash Partition Layout
 - 6.3 OTA Upgrade Flow
 - 6.4 Version Extraction — Magic Number Search Method
 - 6.5 OTA Core APIs
 - 6.6 Firmware Server
 - 6.7 OTA Precautions
7. WiFi Common Troubleshooting

Common Network Communication Protocols

1. TCP/IP Protocol Stack
 - 1.1 Layered Model
 - 1.2 Data Encapsulation Process
 - 1.3 Key Protocol Comparison
2. IP Address
 - 2.1 IPv4 Basics
 - 2.2 Subnet Mask
 - 2.3 Public IP vs Private IP
 - 2.4 Special IP Addresses
3. Port
 - 3.1 Concept of Port
 - 3.2 Client Port vs Server Port
4. Socket Programming Basics
 - 4.1 What is Socket
 - 4.2 Key Data Structures

- 4.3 Network Byte Order
 - 4.4 Socket Creation Types
- 5. TCP In-Depth
 - 5.1 Three-Way Handshake
 - 5.2 Four-Way Wavehand
 - 5.3 How Reliable Transmission is Achieved
 - 5.4 Three Cases of `recv()` Return Value
 - 5.5 TCP Server Standard Flow
- 6. UDP In-Depth
 - 6.1 What "Connectionless" Means
 - 6.2 UDP Applicable Scenarios
 - 6.3 UDP vs TCP API Comparison
 - 6.4 Non-Blocking Receive (MSG_DONTWAIT)
- 7. HTTP Protocol Basics
 - 7.1 Request Format
 - 7.2 Response Format
 - 7.3 Common Status Codes
 - 7.4 Common HTTP Methods
- 8. MQTT Protocol Basics
 - 8.1 Why IoT Uses MQTT Instead of HTTP
 - 8.2 Publish/Subscribe Model
 - 8.3 Topic Wildcards
 - 8.4 QoS Three Levels
- 9. ESP-IDF Network API Selection Guide
- 10. Common Debug Methods
 - 10.1 Troubleshooting Steps
 - 10.2 Common Debug Commands
- 11. Common Network Problems

1. Electromagnetic Waves and Wireless Communication

1.1 What are Electromagnetic Waves

The essence of WiFi is **electromagnetic waves** - changing electric fields generate magnetic fields, changing magnetic fields generate electric fields, and the two alternate to propagate through space.

Key Parameter	Description
Frequency (f)	Oscillations per second, unit Hz. WiFi uses 2.4 GHz = 2.4 billion oscillations per second
Wavelength (λ)	Physical length of one complete wave. $\lambda = c / f \approx 0.125\text{m}$ (~12.5cm at 2.4GHz)
Amplitude	Signal strength, affected by distance and obstacles
Modulation	Method of "encoding" data onto a carrier wave (amplitude/frequency/phase modulation)

The higher the electromagnetic wave frequency, the shorter the wavelength, the weaker the penetration ability (greater wall penetration loss), but the more data it can carry.

1.2 Why WiFi Uses 2.4GHz

Frequency Band	Wavelength	Characteristics
2.4 GHz	~12.5 cm	Strong wall penetration, but crowded spectrum with more interference (microwave ovens, Bluetooth operate on this band)
5 GHz	~6 cm	Faster speed, less interference, but weaker wall penetration
6 GHz (WiFi 6E)	~5 cm	Latest band, extremely many channels, but shortest range

ESP32-S3 **only supports 2.4GHz**. 5G WiFi routers need to enable 2.4G band as well for ESP32-S3 to connect.

2. How WiFi Works

2.1 Channels

The 2.4GHz band is divided into **14 channels** (China uses 1~13), with each channel spaced 5MHz apart, but WiFi signals themselves occupy 22MHz bandwidth.

Channel: 1 6 11
Frequency: 2412MHz 2437MHz 2462MHz
 ■ ■ ← Non-overlapping (recommended to use 1/6/11)

Channel Selection Principle: Adjacent routers should select different channels among 1, 6, 11 to avoid co-channel interference.

2.2 RSSI (Signal Strength)

RSSI (Received Signal Strength Indicator) indicates how strong the signal the receiver "hears":

RSSI Range	Signal Quality	User Experience
> -50 dBm	Excellent	Router is right next to you
-50 ~ -65 dBm	Good	Normal use
-65 ~ -75 dBm	Fair	Possible occasional stuttering
-75 ~ -85 dBm	Weak	Edge area, prone to disconnection
< -85 dBm	Very Weak	Basically unusable

dBm is a logarithmic unit: every 3 dB decrease halves the signal power. **-50 dBm** is 1000 times stronger than **-80 dBm**.

2.3 802.11 Protocol Evolution

Standard	Year	Frequency Band	Max Rate	Common Name
802.11b	1999	2.4 GHz	11 Mbps	—
802.11g	2003	2.4 GHz	54 Mbps	—
802.11n	2009	2.4/5 GHz	150 Mbps	WiFi 4
802.11ac	2013	5 GHz	866 Mbps	WiFi 5
802.11ax	2019	2.4/5/6 GHz	1200 Mbps	WiFi 6

ESP32-S3 supports **802.11 b/g/n**, with theoretical maximum rate of 150 Mbps (HT40 mode), actual throughput ~20~50 Mbps.

2.4 WiFi Security Authentication

Authentication Mode	Encryption Algorithm	Security	ESP32-S3 Support
OPEN	None	Insecure	✓
WEP	RC4	Deprecated (crackable in minutes)	✓
WPA-PSK	TKIP	Weak	✓
WPA2-PSK	AES-CCMP	Secure (Recommended)	✓
WPA3-PSK	SAE	More Secure	✓
WPA2-Enterprise	AES + RADIUS	Enterprise-grade	✓

3. WiFi Operating Modes

3.1 STA Mode (Station)

ESP32 connects to an existing router like a phone, getting an IP address to access the internet.

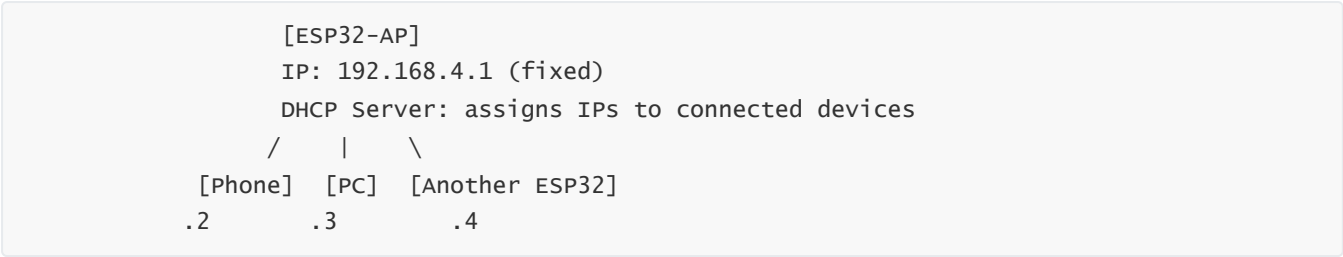


Characteristics:

- IP is assigned by router DHCP (not fixed, unless static IP is set)
- Can access internet and LAN
- Foundation for all subsequent network examples (UDP/TCP/HTTP/MQTT)

3.2 AP Mode (Access Point)

ESP32 **becomes its own hotspot**, other devices connect to it.

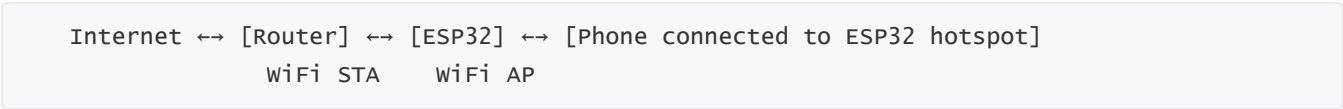


Characteristics:

- IP is fixed at 192.168.4.1, ESP32 acts as DHCP server itself
- Cannot access internet (unless doing NAT forwarding)
- Used for configuration provisioning, direct connection control, offline LAN communication

3.3 STA + AP Hybrid Mode

ESP32 operates as both Station and Access Point simultaneously:



Typical application: **SmartConfig Provisioning** - Phone connects to ESP32 hotspot → tells it the router password → ESP32 switches to STA mode to connect to router.

3.4 STA vs AP Comparison

Comparison	STA Mode	AP Mode
Role	Client (connects to others)	Hotspot (others connect to it)
IP Source	DHCP dynamic assignment	Fixed 192.168.4.1
DHCP Service	Provided by router	Provided by ESP32 itself
Max Connections	N/A	Configurable (default 4, max 10)
Network Capability	Can access internet	LAN only
Core API	<code>esp_wifi_connect()</code>	<code>esp_wifi_start()</code> (no connect needed)
Typical Scenario	IoT device internet access, data reporting	Configuration provisioning, direct control

4. ESP32-S3 WiFi Hardware Specifications

Feature	Parameter
WiFi Standard	IEEE 802.11 b/g/n
Operating Frequency	2.4 GHz
Max Transmit Power	+21 dBm (11b)
Max Receive Sensitivity	-97 dBm (11b)
Supported Protocols	WPA/WPA2/WPA3 Personal/Enterprise
Antenna	On-board ceramic antenna / IPEX external antenna
Operating Modes	STA / AP / STA+AP / Sniffer
Channel Range	1 ~ 14

5. ESP32-S3 WiFi Development Key Points

5.1 WiFi and ADC2 Resource Conflict

ESP32-S3 has two ADC units: ADC1 and ADC2. **ADC2 shares internal hardware resources with WiFi:**

Scenario	ADC1	ADC2
WiFi Off	Normal	Normal
WiFi On	Normal	Abnormal readings / unavailable

In development practice, any project enabling WiFi must use ADC1 (GPIO 1~9 channel range) for analog采集.

5.2 NVS - WiFi Storage Dependency

WiFi driver needs to read RF calibration parameters stored in NVS (Non-Volatile Storage) during initialization. **Regardless of whether WiFi configuration is saved, `nvs_flash_init()` is required.**

WiFi driver depends on RF calibration parameters stored in NVS; NVS partition must be initialized before WiFi starts - if partition is full or version incompatible, just erase and reinitialize.

```
// Initialize NVS flash, WiFi driver depends on RF calibration parameters
esp_err_t ret = nvs_flash_init();
if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    // NVS partition full or version mismatch – erase and retry
    ESP_ERROR_CHECK(nvs_flash_erase());
    ret = nvs_flash_init();
}
```

5.3 WiFi Initialization Fixed Sequence

ESP-IDF WiFi initialization sequence is **strictly fixed** - missing any step causes crash:

WiFi initialization has strict sequence requirements, the following ten steps are all essential, reversed order leads to startup failure.

```
① nvs_flash_init()           ← NVS storage
② esp_netif_init()           ← TCP/IP network stack initialization
③ esp_event_loop_create_default() ← Event loop
④ esp_netif_create_default_wifi_sta() (or _ap()) ← Network interface
⑤ esp_wifi_init(&cfg)         ← WiFi driver
⑥ Register event callbacks    ← WIFI_EVENT + IP_EVENT
⑦ esp_wifi_set_mode()         ← STA / AP / APSTA
⑧ esp_wifi_set_config()       ← SSID + Password
⑨ esp_wifi_start()           ← Start WiFi
⑩ esp_wifi_connect()          ← STA mode initiates connection (AP mode doesn't need this)
```

If `esp_wifi_connect()` is called immediately after `esp_wifi_start()`, it may cause crash, it is recommended to add `vTaskDelay(500ms)` to wait for low-level readiness.

5.4 Antenna

ESP32-S3 development boards usually use **on-board ceramic antenna** (the small gray square on the PCB). Some core boards provide **IPEX socket** for external high-gain antenna:

Antenna Type	Gain	Applicable Scenario
On-board ceramic antenna	~2 dBi	Short range, general use
IPEX external rod antenna	~5 dBi	Wall penetration, long range

When using IPEX external antenna, set antenna selection GPIO via menuconfig or call `esp_wifi_set_ant_gpio()` in code.

5.5 Strapping Pins

ESP32-S3's **GPIO0, GPIO3, GPIO45, GPIO46** are Strapping pins (determine chip operating mode at startup), with internal pull-up/pull-down resistors. **These pins cannot be used for ADC input** because internal resistors will raise/lower the voltage level, making readings always close to VCC or GND.

5.6 WiFi Event-Driven Model

ESP-IDF WiFi uses **event-driven (Event-Driven)** architecture. WiFi state changes generate events from the underlying layer, developers register callback functions to handle:

WiFi underlying state changes trigger corresponding events, callback function dispatches to different handling branches based on event type.

```
wifi state change → WIFI_EVENT / IP_EVENT → event_handler()
├─ WIFI_EVENT_STA_START      → Auto connect
├─ WIFI_EVENT_STA_DISCONNECTED → Clear flag + Reconnect
├─ WIFI_EVENT_SCAN_DONE      → Print AP list
├─ WIFI_EVENT_AP_START       → Hotspot created
├─ WIFI_EVENT_AP_STACONNECTED → Device connected
├─ WIFI_EVENT_AP_STADISCONNECTED → Device disconnected
└─ IP_EVENT_STA_GOT_IP       → Set flag + Print IP
```

5.7 EventGroup Synchronization Mechanism

WiFi connection is **asynchronous** - after `esp_wifi_connect()` call returns immediately, actual connection result is notified via event callback. Using **FreeRTOS Event Group** to implement synchronized waiting:

WiFi connection is an asynchronous operation, main thread blocks waiting for connection result via event group, callback sets bit to unblock.

```
// Main thread: block waiting for WiFi connection success
xEventGroupWaitBits(wifi_event_group,    // Event group handle
                    WIFI_CONNECTED_BIT, // Event bit to wait for
                    // Don't clear the bit on exit
                    pdFALSE,
                    // wait for any bit (not all)
                    pdFALSE,
                    timeout);            // Block timeout

// Got IP, set bit, wake main thread
xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_BIT);
```

6. WiFi OTA Firmware Upgrade

6.1 What is OTA

OTA (Over-The-Air) is one of the most important WiFi applications - device downloads new firmware via WiFi, writes to Flash and restarts to switch, all without USB cable or JTAG programmer.

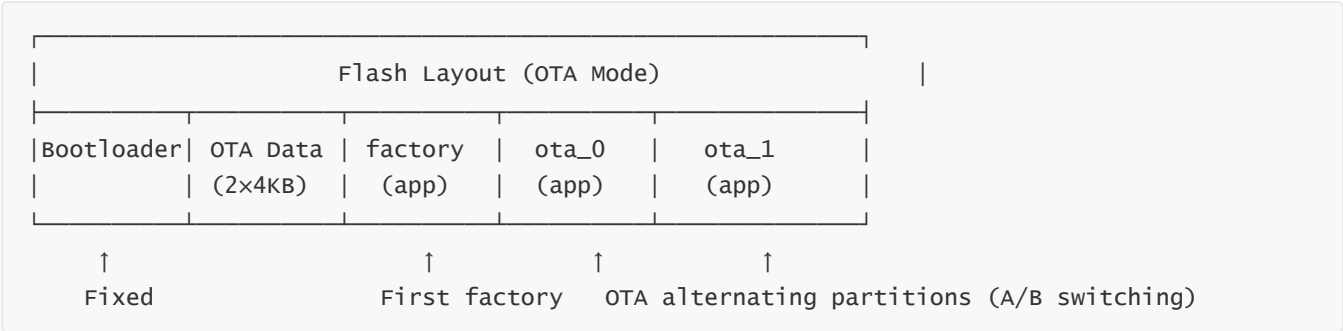
Upgrade Method	Operation	Applicable Stage	Requires Physical Connection
USB Serial Flash	USB cable + idf.py flash	Development/Debugging	Yes
JTAG Flash	JTAG debugger	Development/Debugging	Yes
WiFi OTA	HTTP Download + Partition Switch	Mass Production	No

OTA is a **core operational capability** for IoT products - deployed devices can be repaired without recall.

6.2 Flash Partition Layout

OTA requires at least two APP partitions alternating (A/B switching) to prevent brick from power failure during upgrade:

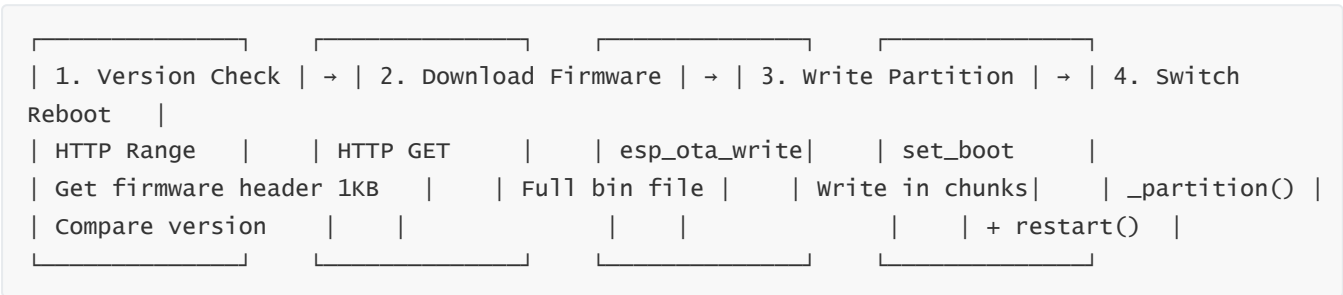
The following is the Flash partition layout in OTA mode, factory stores factory firmware, ota_0 and ota_1 alternate writing new firmware, OTA Data records metadata for next boot partition.



- **factory:** Factory firmware, written during mass production
- **ota_0 / ota_1:** Two partitions alternating use. Currently running on ota_0 → OTA writes to ota_1 → Restart boots from ota_1 → Next OTA writes to ota_0
- **OTA Data:** Records metadata of "which partition to boot from next" (2 sectors backup each other)

Why two APP partitions are needed? If power fails while writing new firmware, old partition is unaffected, device restarts and still runs old version normally. This is OTA's "anti-brick" mechanism.

6.3 OTA Upgrade Flow



Step 1 — Version Check: Use HTTP Range request to download only the first 1024 bytes of firmware, search `esp_app_desc_t` magic number (0xABCD5432) to locate version string, compare with local `esp_app_get_description()->version`. If server version ≤ local version, skip upgrade.

Step 2 — Download Firmware: After confirming new version available, HTTP GET downloads the full `.bin` firmware file.

Step 3 — Write in Chunks: `esp_ota_begin()` opens target partition → loop `esp_http_client_read()` + `esp_ota_write()` → `esp_ota_end()` closes and verifies.

Step 4 — Switch Boot: `esp_ota_set_boot_partition()` modifies OTA Data pointing to new partition → `esp_restart()` reboots.

6.4 Version Extraction — Magic Number Search Method

`esp_app_desc_t` structure layout in bin file:

Firmware header contains magic number identifier and version number field, locate version information by searching fixed byte sequence - magic number `0xABCD5432` at offset `0x00`, 32-byte version string at offset `0x10`.

offset	Content	
0x00	magic_word = 0xABCD5432	← Magic number (little-endian: 32 54 CD AB)
0x04	secure_version	
0x08	Reserved field	
0x10	version[32]	← Version string, e.g., "1.0.0"
...		

Search process:

1. HTTP Range `bytes=0-1023` downloads firmware header 1KB
2. Byte-by-byte match `{0x32, 0x54, 0xCD, 0xAB}`
3. Magic number offset `+0x10` → Read 32-byte version
4. Verify first character is '0'~'9'

6.5 OTA Core APIs

OTA operations follow six-step process: "Get partition → Begin write → Write in chunks → End verification → Set boot partition → Reboot switch", each step corresponds to one core API.

```
// Get "next" writable OTA partition (the one outside current running partition)
const esp_partition_t *esp_ota_get_next_update_partition(NULL);

// Begin OTA write (OTA_SIZE_UNKNOWN = unknown firmware size)
esp_ota_begin(partition, OTA_SIZE_UNKNOWN, &handle);

// Write in chunks (4KB max per chunk)
esp_ota_write(handle, data, size);

// End write (verify + close)
esp_ota_end(handle);

// Set next boot partition (modify OTA Data metadata)
esp_ota_set_boot_partition(partition);

// Reboot to new firmware
esp_restart();
```

6.6 Firmware Server

OTA requires an HTTP file server to host `.bin` files. For testing, you can start a simple server on PC:

PC side executes one command to start HTTP server, ESP32 accesses this address via WiFi to download firmware.

```
# Python 3 one-command HTTP server (port 8080, root = current directory)
python -m http.server 8080
```

```
# On windows, ensure firewall allows port 8080
```

ESP32 side `OTA_URL` points to `http://<PC_IP>:8080/wiFi_OTA.bin`.

6.7 OTA Precautions

Key Point	Description
Partition table must include OTA partitions	menuconfig → Partition Table → "Factory app, two OTA definitions"
Firmware size limit	ota_0/ota_1 each occupy half of Flash minus factory, exceeds causes <code>esp_ota_write</code> failure
Network stability	WiFi disconnects mid-download → <code>esp_ota_abort()</code> rollback, device still runs old version
Version number format	Pure number + dots (<code>1.0.0</code>), don't include <code>v</code> prefix
Security	Production should use HTTPS + certificate verification + firmware signature to prevent man-in-the-middle attacks
Rollback	ESP-IDF supports automatic rollback - if new firmware reboots continuously N times, automatically switches back to old version

For complete code implementation and operation steps, see (5. Network Protocols and Wireless Communication -> 12.WiFi-OTA-Upgrade -> WiFi-OTA-Upgrade).

7. WiFi Common Troubleshooting

Symptom	Cause	Solution
Continuously prints "Disconnected, reconnecting..."	SSID or password incorrect	Check <code>WIFI_SSID</code> and <code>WIFI_PASS</code> macros
Connecting to 5G WiFi fails	ESP32-S3 only supports 2.4G	Ensure router 2.4G band is enabled
Got IP but can't ping external network	Router not connected to internet / DNS issue	Ping router IP first to confirm LAN connectivity

Symptom	Cause	Solution
<code>esp_wifi_connect()</code> returns error	WiFi initialization not complete	Ensure <code>vTaskDelay(500ms)</code> after <code>esp_wifi_start()</code>
<code>WIFI_EVENT_STA_DISCONNECTED</code> triggers frequently	Weak signal or crowded router channel	Move closer to router / Change WiFi channel
Compilation error <code>nvs_flash.h</code> not found	NVS not initialized	Add <code>nvs_flash_init()</code> at the beginning of <code>app_main()</code>
AP mode phone can't find hotspot	Channel occupied / Not started normally	Check logs confirm <code>WiFi AP started</code> ; Try changing <code>AP_CHANNEL</code>
AP mode connection shows "password incorrect"	Password less than 8 characters	WPA2 requires password ≥ 8 characters

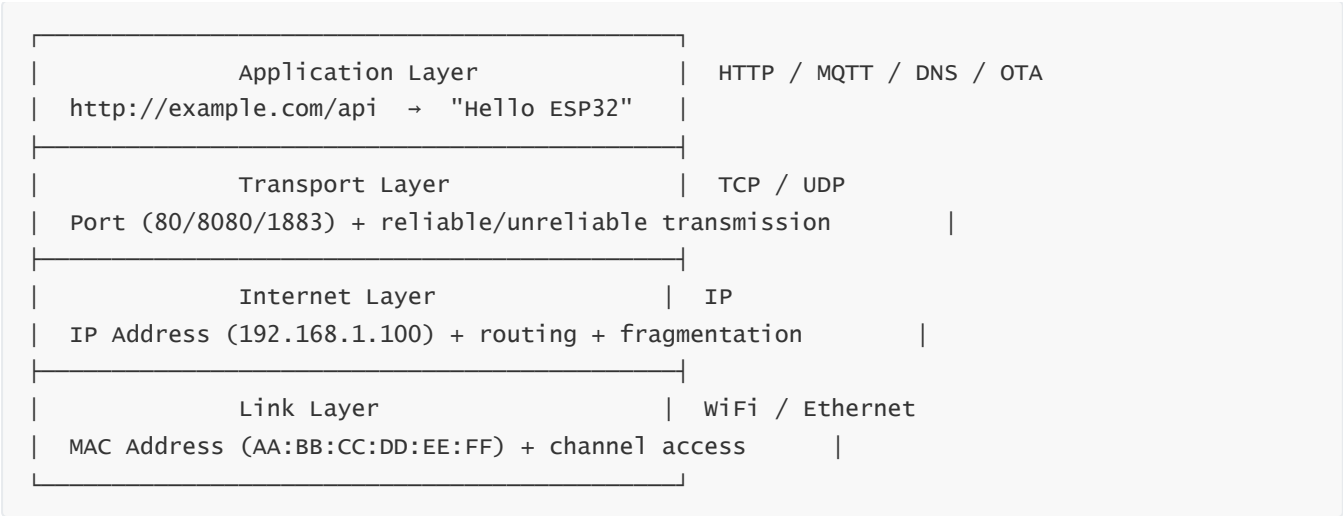
Common Network Communication Protocols

Importance: TCP/IP protocol stack is the fundamental cornerstone of all network communication for ESP32-S3 - whether you use WiFi, Ethernet, HTTP, MQTT or OTA upgrade, data ultimately transmits over the network through TCP/IP protocol stack. Understanding the layered model, IP addressing, port mechanism, Socket API, and core differences between TCP/UDP are prerequisites for troubleshooting network problems, optimizing communication performance, and designing reliable IoT applications. This chapter does not involve specific code flashing, but establishes a **theoretical framework** for network communication - all API calls, parameter configurations, and exception troubleshooting in subsequent network examples (UDP/TCP/HTTP/MQTT) are based on knowledge from this chapter.

1. TCP/IP Protocol Stack

1.1 Layered Model

The essence of network communication is **layer-by-layer encapsulation, peer-to-peer communication**:



1.2 Data Encapsulation Process

When sending data, each layer adds its own header, like **Russian nesting dolls**:

```
Application Layer: [ HTTP GET /index.html ]
                  ↓ Add TCP header
Transport Layer:  [ TCP header | src_port=12345 dst_port=80 | HTTP GET ... ]
                  ↓ Add IP header
Network Layer:   [ IP header | src=192.168.1.100 dst=93.184.216.34 | TCP header | HTTP GET ... ]
                  ↓ Add WiFi frame header
Link Layer:      [ WiFi header | src=AA:BB:CC dst=DD:EE:FF | IP header | TCP header | HTTP GET ... | FCS checksum ]
```

Receiving end unpacks layer by layer until application layer gets raw data.

1.3 Key Protocol Comparison

Protocol	Layer	Reliability	Connection	Header Overhead	Typical Use
IP	Network Layer	Unreliable	Connectionless	20B	Addressing, routing
ICMP	Network Layer	Unreliable	Connectionless	8B	Ping, error reporting
TCP	Transport Layer	Reliable	Connection-oriented	20B	Web, files, MQTT
UDP	Transport Layer	Unreliable	Connectionless	8B	DNS, video streaming, sensors
ARP	Link Layer	—	—	28B	IP→MAC address resolution

Protocol	Layer	Reliability	Connection	Header Overhead	Typical Use
DHCP	Application Layer	—	—	—	Auto IP assignment

2. IP Address

IP address is the "house number" for network communication - after ESP32 gets IP, it can be accessed by other devices on LAN, which is prerequisite for all subsequent Socket communication, HTTP requests, and MQTT connections.

2.1 IPv4 Basics

IPv4 address consists of **32 bits**, written as 4 decimal numbers (each 0~255):

192 . 168 . 1 . 100
11000000 10101000 00000001 01100100 ← 32-bit binary
└──────────────────┬──────────────────┘
Network part Host part
(subnet mask determines boundary)

2.2 Subnet Mask

Subnet mask determines which part of IP address is "network number" and which part is "host number":

Subnet Mask	CIDR	Available IPs	Common Scenario
255.255.255.0	/24	254	Home router
255.255.0.0	/16	65534	Large enterprise

IP: 192.168. 1.100
Mask: 255.255.255. 0
 └───┬───┘
 Network Host (0~255, where 0=network, 255=broadcast)

All devices on the same network have the same "network part", different "host part".

2.3 Public IP vs Private IP

Range	Type	Description
10.0.0.0 ~ 10.255.255.255	Private Class A	Large enterprise internal
172.16.0.0 ~ 172.31.255.255	Private Class B	Medium enterprise internal
192.168.0.0 ~ 192.168.255.255	Private Class C (Home Ethernet)	Most common - home router

Range	Type	Description
Others	Public IP	Unique on internet, requires purchase

Private IP cannot be accessed directly from internet - router maps private IP to public IP through **NAT** (Network Address Translation).

2.4 Special IP Addresses

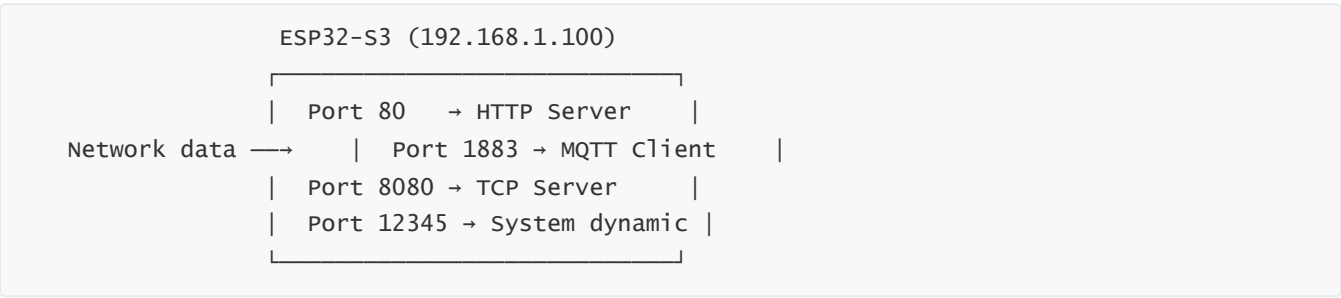
IP	Meaning	Usage
127.0.0.1	Localhost	Access yourself
0.0.0.0	Any address	Bind to all network interfaces
255.255.255.255	Broadcast address	Send to all devices on LAN
192.168.1.1	Usually router gateway	Management page entry

3. Port

Port is the key to achieving "one device, multiple services" - ESP32 can simultaneously run HTTP server (80), MQTT client (1883), and TCP data channel (8080), each service binding different ports without interference. Firewall and NAT configurations also depend on port numbers.

3.1 Concept of Port

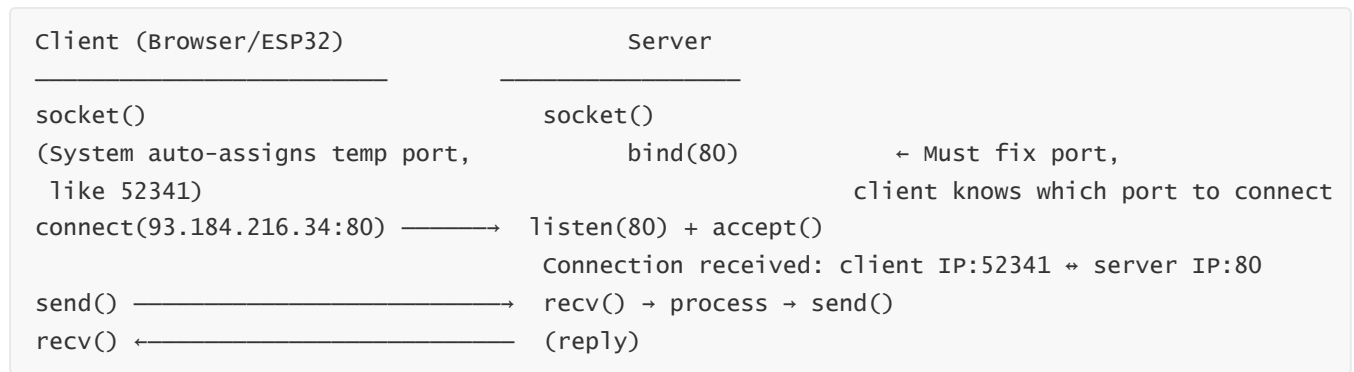
IP address locates a device, port locates a program on that device.



Port is a 16-bit number (0~65535):

Range	Type	Description
0 ~ 1023	Well-known ports	HTTP=80, HTTPS=443, MQTT=1883, DNS=53
1024 ~ 49151	Registered ports	Custom services (e.g., 8080)
49152 ~ 65535	Dynamic ports	Client temporary use

3.2 Client Port vs Server Port



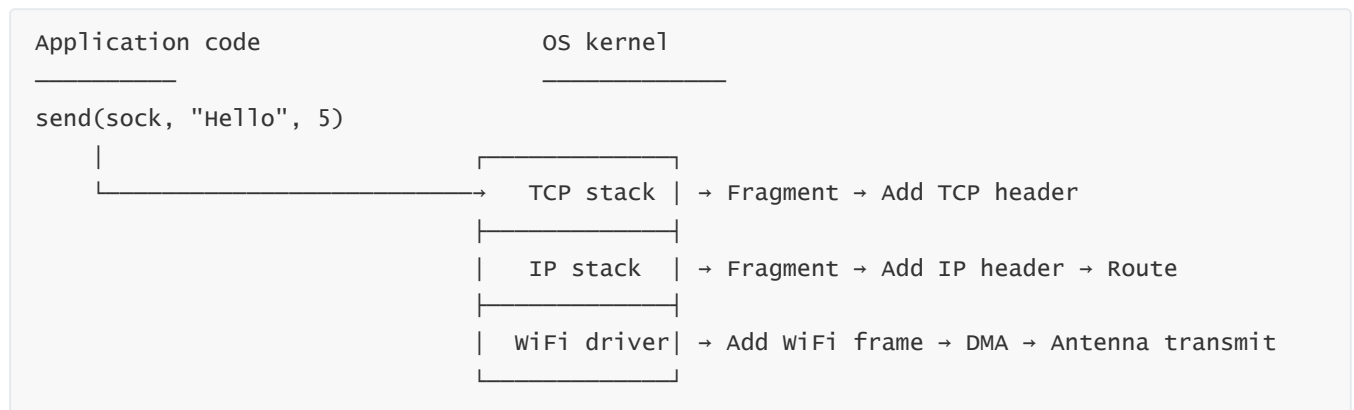
Server must `bind()` fixed port, otherwise client doesn't know where to connect; client port is auto-assigned by OS (discarded after use).

4. Socket Programming Basics

Socket is the **core API layer** for network programming in ESP-IDF - whether custom TCP/UDP communication, HTTP client or MQTT client, all underlying data is sent/received via Socket. Mastering Socket data structures, byte order conversion, and creation methods is the foundation for reading and writing all network code.

4.1 What is Socket

Socket is a network programming interface provided by the operating system - developers send/receive data through Socket without caring about underlying WiFi frames, IP fragmentation, TCP retransmission and other details.



4.2 Key Data Structures

Two most important address structures in Socket programming - `sockaddr` is generic type, `sockaddr_in` is IPv4-specific type, all Socket APIs use latter via casting.

```
// Generic Socket address (supports IPv4/IPv6)
struct sockaddr {
    // Address family: AF_INET(IPv4) or AF_INET6(IPv6)
    sa_family_t sa_family;
    char        sa_data[14]; // Protocol address (IP + Port)
};
```



```

struct sockaddr_in {
    sa_family_t    sin_family; // Address family: IPv4
    // Port number (network byte order)
    uint16_t       sin_port;
    struct in_addr sin_addr;    // IP address
};
struct sockaddr_in server_addr;
bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

```

4.3 Network Byte Order

Network byte order = Big-Endian - high-order byte stored at low address. ESP32 CPU is little-endian, opposite of network order, must convert:

Number 0x12345678 (32-bit)

Big-Endian (Network Order):

Low address ← → High address

12 | 34 | 56 | 78 ← High byte first

Little-Endian (ESP32 Host Order):

Low address ← → High address

78 | 56 | 34 | 12 ← Low byte first

Function	Meaning	Usage
<code>htons(n)</code>	Host TO Network Short (16-bit)	Port conversion
<code>htonl(n)</code>	Host TO Network Long (32-bit)	IP address conversion (less used)
<code>ntohs(n)</code>	Network TO Host Short	Restore after receiving
<code>ntohl(n)</code>	Network TO Host Long	Restore after receiving
<code>inet_pton()</code>	String → Binary IP (auto network order)	"192.168.1.1" → 0x0101A8C0

Ports and IP addresses need conversion to network byte order - `htons()` for ports, `inet_pton()` converts IP string to network-order binary.

```

// Correct approach: use conversion functions
// Convert to network byte order
addr.sin_port = htons(8080);
// String → Binary (network order)
inet_pton(AF_INET, "192.168.1.100", &addr.sin_addr);

// Wrong approach: direct assignment
// ❌ CPU little-endian, network sees 0x901F
addr.sin_port = 8080;

```

4.4 Socket Creation Types

`socket()` distinguishes TCP (stream) from UDP (datagram) via `type` parameter - this determines communication mode for all subsequent Socket operations.

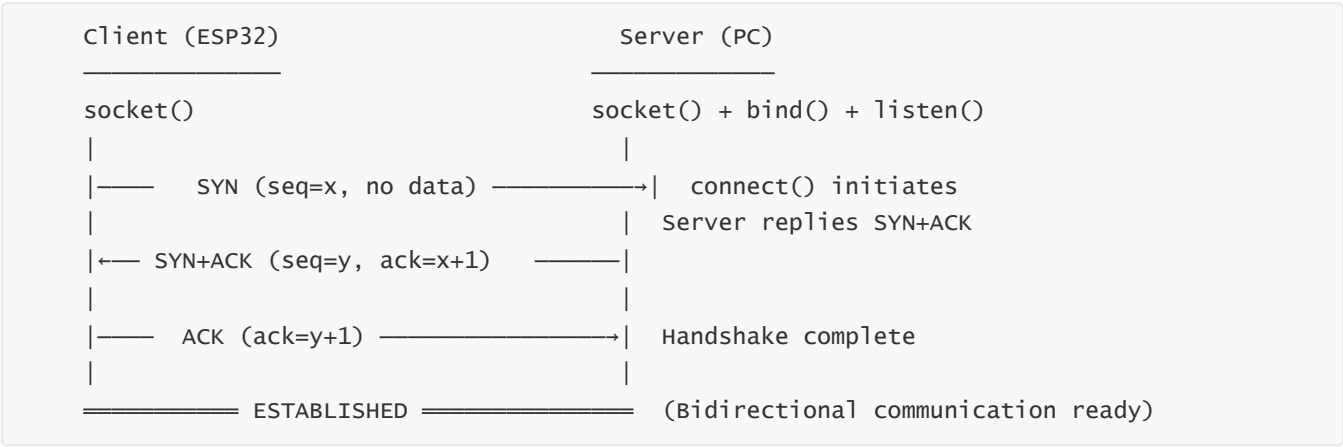
```
int fd = socket(domain, type, protocol); // General form
int tcp_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
int udp_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
```

Parameter	TCP	UDP
domain	AF_INET (IPv4)	AF_INET (IPv4)
type	SOCK_STREAM (stream)	SOCK_DGRAM (datagram)
Behavior	Connection-oriented, reliable, byte stream	Connectionless, unreliable, preserves message boundaries

5. TCP In-Depth

TCP is the most commonly used reliable transmission protocol for IoT devices - firmware OTA upgrades cannot lose a single byte, MQTT messages must be reliably delivered, HTTP pages must load completely, all these scenarios rely on TCP's acknowledgment and timeout retransmission mechanism. Understanding three-way handshake, four-way wave (connection termination), and `recv()` return values is the foundation for stable TCP server operation.

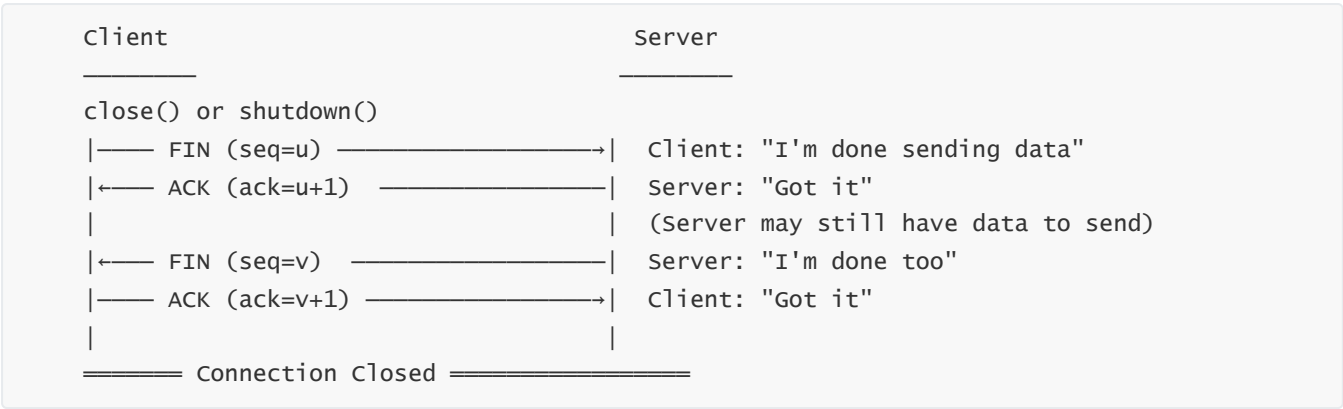
5.1 Three-Way Handshake



- SYN: Request to establish connection
- SYN+ACK: Agree + Request (server also confirms its direction)
- ACK: Acknowledge (formally established)

The essence of three-way handshake is **each party confirms the other's send/receive capability is normal**.

5.2 Four-Way Wavehand



TCP is full-duplex - each direction closes independently. `close()` closes both directions; `shutdown(fd, SHUT_WR)` only closes write direction (can still receive).

5.3 How Reliable Transmission is Achieved

TCP's reliability comes from three mechanisms:

Mechanism	Principle
Acknowledgment (ACK)	Reply ACK after receiving data, sender retransmits if no ACK received
Sequence Number (SEQ)	Each byte has a number, receiver reassembles in order
Timeout Retransmission (RTO)	Timer starts after sending, retransmit if ACK not received within timeout

5.4 Three Cases of `recv()` Return Value

Return Value	Meaning	Handling
> 0	Received n bytes of data	Normal processing
= 0	Peer closed connection normally (sent FIN)	<code>close()</code> and <code>break</code>
< 0	No data yet (<code>MSG_DONTWAIT</code> mode) or error	Check <code>errno</code>

`recv()` three return values correspond to three handling branches - this is the standard template for TCP receive logic.

```
int ret = recv(sock_fd, buf, len, 0);
if (ret > 0) { /* Data received, process normally */ }
else if (ret == 0) { /* Peer closed, break + close() */ }
else { /* No data (non-blocking), or real error */ }
```

5.5 TCP Server Standard Flow

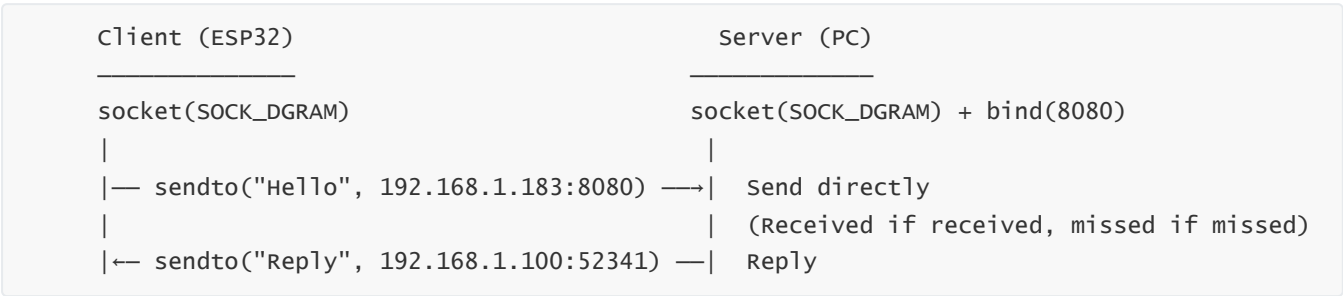
```
socket() → bind() → listen() → while(1) {
  client_fd = accept() → while(1) {
    recv() / send()    ← Communicate with single client
  } → close(client_fd) ← Client disconnected, return to accept
}
```

6. UDP In-Depth

UDP's low overhead and low latency characteristics make it the choice for high-frequency sensor data reporting, real-time audio/video streaming, DNS queries and other scenarios. In IoT, when "speed takes priority over reliability" - such as temperature/humidity collection 100+ times per second, device discovery broadcasts - UDP is more suitable than TCP.

6.1 What "Connectionless" Means

UDP doesn't handshake, doesn't establish connection, doesn't guarantee delivery - datagrams are like **throwing a message in a bottle (drift bottle)**:



6.2 UDP Applicable Scenarios

✔ Suitable for UDP	✗ Not Suitable for UDP
Video/audio real-time streaming (occasional glitches acceptable)	File transfer (can't lose a single byte)
DNS queries (small request volume, timeout retry works)	Web browsing (HTML must be complete)
High-frequency sensor data (losing a few frames is fine)	Firmware OTA upgrade (firmware must be complete)
Broadcast/multicast messages	Financial transactions (packet loss = money loss)

6.3 UDP vs TCP API Comparison

TCP vs UDP API call flow comparison - TCP requires `listen()` + `accept()` to establish connection then communicates via file descriptor, UDP directly uses `sendto()` / `recvfrom()` with target address.

```
// ===== TCP Server ===== // ===== UDP Server =====
socket(AF_INET, SOCK_STREAM, 0)      socket(AF_INET, SOCK_DGRAM, 0)
bind(fd, addr, len)                  bind(fd, addr, len)
listen(fd, 5)                        // No listen
// No accept, use recvfrom directly
client_fd = accept(fd, ...)
recv(client_fd, buf, len, 0)          recvfrom(fd, buf, len, 0, &addr, &len)
send(client_fd, buf, len, 0)          sendto(fd, buf, len, 0, &addr, len)

// ===== TCP Client ===== // ===== UDP Client =====
socket(AF_INET, SOCK_STREAM, 0)      socket(AF_INET, SOCK_DGRAM, 0)
connect(fd, &addr, len)               // No connect
send(fd, buf, len, 0)                 sendto(fd, buf, len, 0, &addr, len)
recv(fd, buf, len, 0)                 recvfrom(fd, buf, len, 0, &addr, &len)
```

6.4 Non-Blocking Receive (MSG_DONTWAIT)

`MSG_DONTWAIT` flag makes `recvfrom()` / `recv()` **non-blocking** - returns -1 immediately (`errno=EAGAIN`) when no data, without blocking task, suitable for high-frequency polling scenarios (e.g., 10ms interval low-latency receive loop).

```
int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                        MSG_DONTWAIT, // Non-blocking flag
                        (struct sockaddr *)&from_addr, &len);
if (recv_len > 0) {
    // Data received, process it
}
```

7. HTTP Protocol Basics

HTTP is the standard protocol for IoT devices to interact with cloud REST APIs - devices use GET to fetch configurations, POST to report sensor data, Range requests to implement OTA firmware chunked download. Understanding request/response format and status codes is prerequisite for calling Web APIs.

7.1 Request Format

```
GET /index.html HTTP/1.1\r\n
Host: example.com\r\n
User-Agent: ESP32\r\n
Accept: */*\r\n
\r\n
```

- ← Request line: method + URI + version
- ← Request headers: key-value pairs
- ← Empty line = end of headers
- ← Request body (empty for GET, has content for POST)

7.2 Response Format

HTTP/1.1 200 OK\r\n	← Status line: version + status code + reason phrase
Content-Type: text/html\r\n	← Response headers
Content-Length: 125\r\n	
Connection: close\r\n	
\r\n	← Empty line = end of headers
<html>...response body...</html>	← Response body

7.3 Common Status Codes

Status Code	Meaning	Description
200	OK	Request successful
206	Partial Content	Range request returns partial content (OTA firmware header download)
301	Moved Permanently	Permanent redirect
302	Found	Temporary redirect
400	Bad Request	Request format error
404	Not Found	Resource does not exist
500	Internal Server Error	Server internal error

7.4 Common HTTP Methods

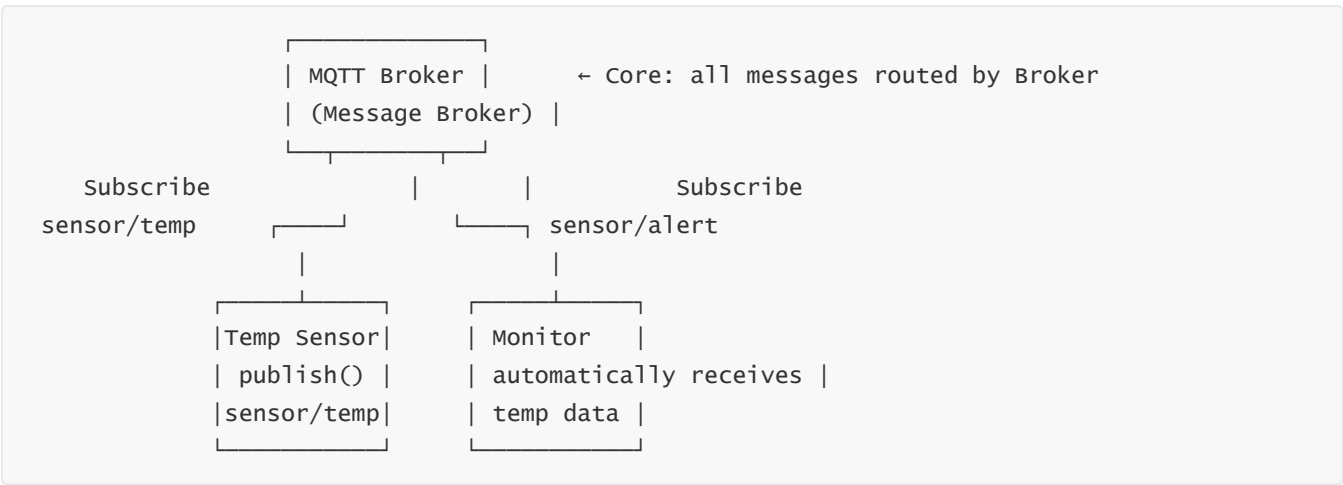
Method	Semantics	Request Body	Typical Use
GET	Get resource	None	Query data, fetch webpage
POST	Submit data	Has	Submit forms, upload sensor data
PUT	Replace resource	Has	Update configuration
DELETE	Delete resource	None	Delete record

8. MQTT Protocol Basics

8.1 Why IoT Uses MQTT Instead of HTTP

Comparison	MQTT	HTTP
Communication Model	Publish/Subscribe (Broker relay)	Request/Response (direct)
Message Push	Native support, subscribe = push	Requires client polling (or WebSocket)
Min Protocol Header	2 bytes	Hundreds of bytes
Connection Keep-alive	Keep Alive heartbeat	Stateless (each request independent connection)
One-to-Many	Naturally supported (multiple people subscribe same Topic)	Requires server polling + push logic

8.2 Publish/Subscribe Model



Three Roles:

Role	Description	Example
Publisher	Sends message to specific Topic	ESP32 reports temperature
Subscriber	Subscribes to Topic, automatically receives messages	Phone App subscribes to sensor data
Broker	Relays all messages, manages Topic subscription relationships	EMQX, Mosquitto

8.3 Topic Wildcards

Wildcard	Semantics	Example
<code>+</code>	Match single level	<code>home/+/light</code> → <code>home/bedroom/light</code>
<code>#</code>	Match multiple levels	<code>home/#</code> → <code>home/bedroom/light/status</code>

8.4 QoS Three Levels

QoS	Semantics	Mechanism
0	At most once (fire and forget)	No retry, no acknowledgment
1	At least once (may duplicate)	Send → Wait for PUBACK → Timeout retransmit
2	Exactly once (no duplicates)	PUBREC→PUBREL→PUBCOMP four-way handshake

9. ESP-IDF Network API Selection Guide

When developing network applications, ESP-IDF provides **from low-level to high-level** APIs, select based on requirements:

```
High-level (simple, well-encapsulated)
| esp_http_client / esp_http_server    ← HTTP request/server
| mqtt_client                          ← MQTT client
| esp_ota                              ← OTA upgrade
|
| lwip/sockets (BSD Socket API)        ← TCP/UDP raw communication
|   socket / bind / listen / accept / connect
|   send / recv / sendto / recvfrom
|
Low-level (flexible, need manual handling)
| esp_wifi                             ← WiFi connect/scan/config
| esp_netif                           ← Network interface management
| esp_event                           ← Event-driven framework
```

If you want to do...	Use this API
Connect to WiFi	<code>esp_wifi</code> + <code>esp_netif</code>
Custom TCP/UDP communication	<code>lwip/sockets</code> (BSD Socket)
Access webpage / REST API	<code>esp_http_client</code>
Build Web server	<code>esp_http_server</code>
IoT sensor reporting / remote control	<code>mqtt_client</code>
WiFi remote firmware upgrade	<code>esp_ota</code> + <code>esp_http_client</code>

If you want to do...	Use this API
Ping remote host	<code>lwip/ping.h</code> or <code>esp_ping</code>

10. Common Debug Methods

Network problem troubleshooting uses **layer-by-layer localization** strategy - confirm from bottom to top: WiFi connection → IP obtained → LAN connectivity → Port listening → Application layer protocol. Skipping levels easily leads to confusion.

10.1 Troubleshooting Steps

Problem: ESP32 cannot communicate

1. First check if serial logs show "Got IP: xxx.xxx.xxx.xxx"
 - └ No → WiFi not connected → Check SSID/password/frequency band
 - └ Yes → WiFi normal, continue troubleshooting
2. Ping test
 - └ Ping ESP32 IP → Verify IP layer connectivity
 - └ Ping router IP → Verify LAN connectivity
 - └ Ping 8.8.8.8 → Verify internet connectivity (DNS unrelated, pure IP)
3. Port test
 - └ On PC use `netstat -an | findstr 8080` to check if port is listening
 - └ Use network debug tool to send data to ESP32 IP:PORT
4. Firewall check
 - └ windows firewall may block non-standard ports like 8080

10.2 Common Debug Commands

PC side and ESP32 side commonly used network debug commands - first confirm network connectivity from PC side, then confirm running status from ESP32 logs.

```
# PC side (Windows PowerShell)
ipconfig                # Check local IP
ping 192.168.1.100      # Ping ESP32
netstat -an | findstr 8080 # Check port 8080 status
curl http://192.168.1.100/ # Test HTTP server

# ESP32 side (add in code)
ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&ip)); // Print obtained IP
// Check free memory
ESP_LOGI(TAG, "Free heap: %lu", esp_get_free_heap_size());
```

11. Common Network Problems

Symptom	Cause	Solution
<code>connect()</code> returns -1	Server not started or IP incorrect	Confirm PC side Server is listening, IP and port correct
<code>bind()</code> fails	Port already in use	Change port number / Wait 2 minutes (TIME_WAIT timeout)
<code>recv()</code> returns 0	Peer actively closed connection	Normal disconnect, <code>break</code> then <code>close()</code>
<code>recv()</code> data "sticks together"	TCP is byte stream, no message boundaries	Application layer defines frame header+length or delimiter protocol
<code>send()</code> continuous failures	Peer closed / network interrupted	TCP connection disconnected, need to <code>connect()</code> again
Chinese text garbled	Terminal encoding mismatch	Check serial assistant and network debug tool encoding settings (UTF-8)
PC cannot receive ESP32 data	ESP32 IP changed / firewall blocked	Check ESP32 actual IP via serial, PC firewall allows corresponding port
HTTP client <code>HTTP request error</code>	URL unreachable or DNS resolution failed	Ensure ESP32 can access internet; or use IP instead of domain name
MQTT <code>MQTT_EVENT_CONNECTED</code> doesn't trigger	Broker address incorrect or WiFi no internet	<code>ping broker.emqx.io</code> to test internet connectivity